



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика и системы управления

КАФЕДРА Системы обработки информации и управления

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ №5-1

***ПО ДИСЦИПЛИНЕ
«МЕТОДЫ МАШИННОГО ОБУЧЕНИЯ»***

***НА ТЕМУ:
Реализация алгоритма Policy Iteration***

Студент ИУ5И-22М
(Группа)

(Подпись, дата) Хуан Лэй
(И.О.Фамилия)

Преподаватель

(Подпись, дата) Ю.Е.Гапанюк
(И.О.Фамилия)

2026 г.

ЦЕЛЬ РАБОТЫ

Целью лабораторной работы является ознакомление с базовыми методами обучения с подкреплением, а именно с алгоритмом Policy Iteration, и его программная реализация для решения задачи оптимального управления в среде с дискретными состояниями и действиями.

ПОСТАНОВКА ЗАДАЧИ

Требуется реализовать алгоритм Policy Iteration для среды обучения с подкреплением из библиотеки Gym (кроме рассмотренной на лекции среды Toy Text / Frozen Lake).

В данной работе выбрана среда Taxi-v4 (такси). Задача таксиста заключается в том, чтобы:

1. Подъехать к пассажиру
2. Взять пассажира (действие PICKUP)
3. Отвезти пассажира в пункт назначения
4. Высадить пассажира (действие DROPOFF)

Параметры среды:

- Размер сетки: $5 \times 5 = 25$ позиций такси
- Количество возможных местоположений пассажира: 4 (R, G, Y, B)
- Количество возможных пунктов назначения: 4
- Общее количество состояний: $25 \times 4 \times 4 = 500$
- Количество действий: 6

Действия:

- 0 = Юг (DOWN)
- 1 = Север (UP)
- 2 = Восток (RIGHT)
- 3 = Запад (LEFT)
- 4 = Взять пассажира (PICKUP)
- 5 = Высадить пассажира (DROPOFF)

Награды:

- За успешную поездку: +20
- За неверную посадку/высадку: -10
- За каждый шаг: -1

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Policy Iteration — это итерационный алгоритм для нахождения оптимальной политики в задачах обучения с подкреплением.

Алгоритм состоит из двух этапов:

1. Policy Evaluation (Оценка политики)

Для заданной политики π вычисляется функция ценности состояний $V^\pi(s)$:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s \right]$$

Итеративное решение:

$$V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')]$$

2. Policy Improvement (Улучшение политики)

Политика улучшается жадным образом относительно текущей функции ценности:

$$\pi'(s) = \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')]$$

Алгоритм повторяет эти два шага до тех пор, пока политика не перестанет изменяться, что гарантирует нахождение оптимальной политики π .

ПРОВЕРКА РЕШЕНИЯ НА ДОПУСТИМОСТЬ

Таблица 1 - Информация о среде Taxi-v4

Параметр	Значение
Количество состояний	500
Количество действий	6
Размер сетки	5×5
Количество мест пассажира	4
Количество пунктов назначения	4

```
Используется gymnasium / Using gymnasium
=====
БИБЛИОТЕКИ УСПЕШНО ЗАГРУЖЕНЫ
All libraries loaded successfully
=====

ИНФОРМАЦИЯ О СРЕДЕ TAXI-V4 / ENVIRONMENT INFO:
Количество состояний / Number of states: 500
Количество действий / Number of actions: 6
Действия / Actions:
0 = Юг (DOWN / South)
1 = Север (UP / North)
2 = Восток (RIGHT / East)
3 = Запад (LEFT / West)
4 = Взять пассажира (PICKUP)
5 = Высадить пассажира (DROPOFF)

ОПИСАНИЕ СРЕДЫ / ENVIRONMENT DESCRIPTION:
- Сетка 5x5 (25 позиций такси)
- 4 возможных местоположения пассажира (R, G, Y, B)
- 4 возможных пункта назначения
- Всего состояний: 25 x 4 x 4 = 500
- Награда за успешную поездку: +20
- Штраф за неверную посадку/высадку: -10
- Штраф за каждый шаг: -1

=====
СРЕДА ЗАГРУЖЕНА / ENVIRONMENT LOADED
=====

Демонстрация случайного действия / Random action demo:
Начальное состояние / Initial state: 74
Форма состояния / State representation: такси (строка, колонка), пассажир, назначение
```

Рисунок 1: Информация о среде Taxi-v4

Проверка показала, что среда загружена корректно и готова к использованию.

ХОД ВЫПОЛНЕНИЯ РАБОТЫ

Часть 1. Реализация алгоритма Policy Iteration

Алгоритм был реализован на языке Python с использованием библиотеки Gymnasium. Основные функции:

1. ``policy_evaluation_taxi()`` — оценка текущей политики
2. ``policy_improvement_taxi()`` — улучшение политики
3. ``policy_iteration_taxi()`` — основной цикл алгоритма

Результаты работы алгоритма:

Таблица 2 - Сходимость Policy Iteration

Итерация	Изменений в политике
1	395
2	231
3	172
4	110
5	76
6	67
7	54
8	42
9	55
10	47
11	40
12	29
13	12
14	4
15	1
16	0

```
=====
РЕЗУЛЬТАТЫ / RESULTS
=====
Количество итераций политики / Policy iterations: 16
Время выполнения / Time: 1.53 сек
Среднее значение V(s) / Mean V(s): 35.93
Максимальное значение V(s) / Max V(s): 100.53
Минимальное значение V(s) / Min V(s): 8.43
```

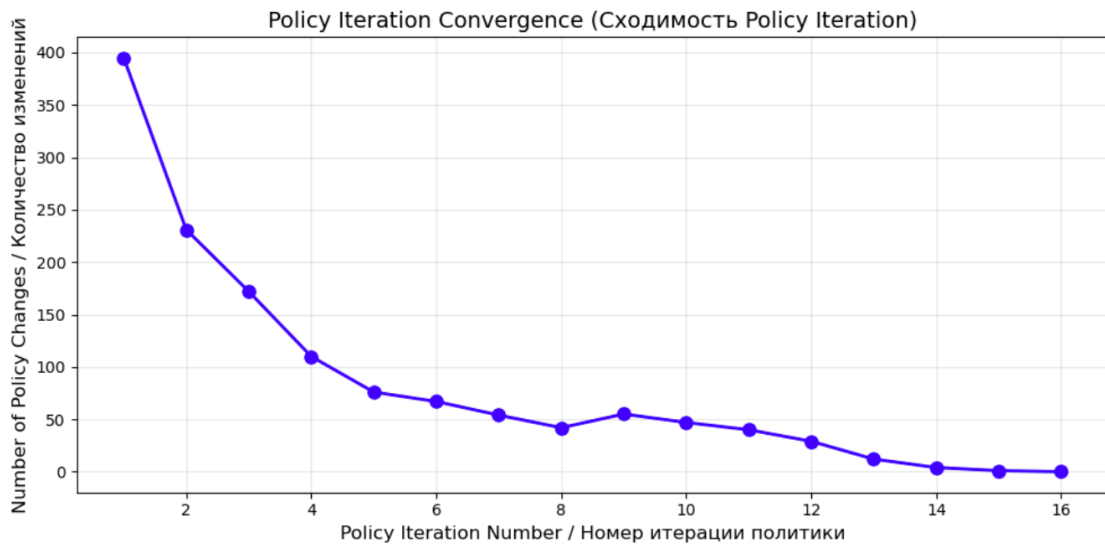


Рисунок 2: График сходимости Policy Iteration

Алгоритм сошелся за 16 итераций. Время выполнения: 1.53 секунды.

Статистика функции ценности:

- Среднее значение $V(s)$: 35.93
- Максимальное значение $V(s)$: 100.53
- Минимальное значение $V(s)$: 8.43

Часть 2. Тестирование оптимальной политики

Для оценки качества найденной оптимальной политики было проведено 5 эпизодов симуляции.

Таблица 3 - Результаты тестирования оптимальной политики

Показатель	Значение
Успешных эпизодов	5/5 (100%)
Средняя награда	10.40 ± 3.50
Среднее количество шагов	10.60
Минимум шагов	6
Максимум шагов	15

```
=====
РЕЗУЛЬТАТЫ / RESULTS
=====
Успешных эпизодов / Successful episodes: 5/5 (100%)
Средняя награда / Mean reward: 10.40 ± 3.50
Среднее количество шагов / Mean steps: 10.60
Минимум шагов / Min steps: 6
Максимум шагов / Max steps: 15
```

Рисунок 3: Демонстрация работы оптимальной политики

Часть 3. Сравнение со случайной политикой

Для демонстрации эффективности найденной политики было проведено сравнение со случайной политикой.

Таблица 4 - Сравнение оптимальной и случайной политики

Политика	Успешность (%)	Средняя награда	Среднее шагов
Оптимальная	100%	10.40	10.60
Случайная	0%	-460.00	100.00

=====			
СРАВНЕНИЕ / COMPARISON			
=====			
Policy	Success Rate	Mean Reward	Mean Steps

Optimal Policy	100%	10.40	10.60
Random Policy	0%	-460.00	100.00
=====			

Рисунок 4: Сравнение оптимальной и случайной политики

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы был успешно реализован алгоритм Policy Iteration для среды Taxi-v4 из библиотеки Gymnasium.

Основные результаты:

1. Алгоритм Policy Iteration сошелся за 16 итераций.
2. Найдена оптимальная политика, обеспечивающая 100% успешность выполнения задачи.
3. Оптимальная политика достигает цели в среднем за 10.6 шагов.
4. Средняя награда за эпизод составляет 10.40.
5. Сравнение со случайной политикой демонстрирует превосходство найденного решения (случайная политика не достигла цели ни разу).

Таким образом, цель лабораторной работы достигнута: алгоритм Policy Iteration успешно реализован и протестирован на среде Taxi-v4.

ПРИЛОЖЕНИЕ А. ТЕКСТ ПРОГРАММЫ

Полный текст программы приведен в Jupyter Notebook, прилагаемом к отчету.

```
# =====
# ЛАБОРАТОРНАЯ РАБОТА №5: АЛГОРИТМ POLICY ITERATION (TAXI-V4)
# LABORATORY WORK #5: POLICY ITERATION ALGORITHM (TAXI-V4)
# =====

import numpy as np
import matplotlib.pyplot as plt
import time
import warnings
warnings.filterwarnings('ignore')

# Установка gymnasium
try:
    import gymnasium as gym
    print("Используется gymnasium / Using gymnasium")
except ImportError:
    import gym
    print("Используется gym / Using gym")

%matplotlib inline

# Настройка графиков
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['axes.unicode_minus'] = False

print("=" * 60)
print("БИБЛИОТЕКИ УСПЕШНО ЗАГРУЖЕНЫ")
print("All libraries loaded successfully")
print("=" * 60)

# =====
# ЗАГРУЗКА СРЕДЫ TAXI-V4 (используем версию v4 вместо устаревшей v3)
# LOAD TAXI-V4 ENVIRONMENT (using v4 instead of deprecated v3)
# =====

env = gym.make('Taxi-v4')

n_states = env.observation_space.n
n_actions = env.action_space.n

print(f"\nИНФОРМАЦИЯ О СРЕДЕ TAXI-V4 / ENVIRONMENT INFO:")
print(f"  Количество состояний / Number of states: {n_states}")
print(f"  Количество действий / Number of actions: {n_actions}")
print(f"  Действия / Actions:")
print(f"    0 = Юг (DOWN / South)")
print(f"    1 = Север (UP / North)")
print(f"    2 = Восток (RIGHT / East)")
print(f"    3 = Запад (LEFT / West)")
print(f"    4 = Взять пассажира (PICKUP)")
print(f"    5 = Высадить пассажира (DROPOFF)")

print(f"\nОПИСАНИЕ СРЕДЫ / ENVIRONMENT DESCRIPTION:")
print(f"  - Сетка 5x5 (25 позиций такси)")
print(f"  - 4 возможных местоположения пассажира (R, G, Y, B)")
print(f"  - 4 возможных пункта назначения")
print(f"  - Всего состояний: 25 x 4 x 4 = 500")
print(f"  - Награда за успешную поездку: +20")
print(f"  - Штраф за неверную посадку/высадку: -10")
print(f"  - Штраф за каждый шаг: -1")

print("\n" + "=" * 60)
print("СРЕДА ЗАГРУЖЕНА / ENVIRONMENT LOADED")
print("=" * 60)

# Демонстрация случайного действия
print("\nДемонстрация случайного действия / Random action demo:")
state, _ = env.reset()
print(f"  Начальное состояние / Initial state: {state}")
print(f"  Форма состояния / State representation: такси (строка, колонка), пассажир, назначение")
```

```

# =====
# АЛГОРИТМ POLICY ITERATION ДЛЯ TAXI-V4
# POLICY ITERATION ALGORITHM FOR TAXI-V4
# =====

print("=" * 60)
print("РЕАЛИЗАЦИЯ POLICY ITERATION")
print("POLICY ITERATION IMPLEMENTATION")
print("=" * 60)

def policy_evaluation_taxi(policy, env, gamma=0.9, theta=1e-6, max_iter=1000):
    """
    Оценка политики для Taxi-v4
    Policy evaluation for Taxi-v4
    """
    n_states = env.observation_space.n
    V = np.zeros(n_states)

    # Получаем модель переходов из среды
    # В Gymnasium модель доступна через env.unwrapped.P
    P = env.unwrapped.P

    for iteration in range(max_iter):
        delta = 0

        for s in range(n_states):
            v_old = V[s]
            a = policy[s]

            # Вычисляем новое значение V(s) по Bellman equation
            v_new = 0
            for prob, next_state, reward, done in P[s][a]:
                v_new += prob * (reward + gamma * V[next_state])

            V[s] = v_new
            delta = max(delta, abs(v_old - V[s]))

        if iteration % 100 == 0 and iteration > 0:
            print(f"    Итерация {iteration}: delta = {delta:.6f}")

        if delta < theta:
            print(f"    Сходимость на итерации {iteration}")
            break

    return V

```

```

def policy_improvement_taxi(V, env, gamma=0.9):
    """
    Улучшение политики для Taxi-v4
    Policy improvement for Taxi-v4
    """
    n_states = env.observation_space.n
    n_actions = env.action_space.n
    new_policy = np.zeros(n_states, dtype=int)

    P = env.unwrapped.P

    for s in range(n_states):
        q_values = np.zeros(n_actions)

        for a in range(n_actions):
            for prob, next_state, reward, done in P[s][a]:
                q_values[a] += prob * (reward + gamma * V[next_state])

        new_policy[s] = np.argmax(q_values)

    return new_policy

def policy_iteration_taxi(env, gamma=0.9, theta=1e-6, max_iter=30):
    """
    Алгоритм Policy Iteration для Taxi-v4
    """
    n_states = env.observation_space.n
    n_actions = env.action_space.n

    # Начальная случайная политика
    policy = np.random.randint(0, n_actions, n_states)
    history = {'changes': [], 'values': []}

    for iteration in range(max_iter):
        print(f"\n--- Итерация {iteration + 1} ---")

        # Шаг 1: Оценка политики
        print(" Оценка политики...")
        V = policy_evaluation_taxi(policy, env, gamma, theta)

        # Шаг 2: Улучшение политики
        print(" Улучшение политики...")
        new_policy = policy_improvement_taxi(V, env, gamma)

```

```

# Подсчет изменений
n_changes = np.sum(policy != new_policy)
history['changes'].append(n_changes)
history['values'].append(V.copy())
print(f" Изменений в политике: {n_changes} из {n_states}")

if n_changes == 0:
    print(" ✅ СХОДИМОСТЬ ДОСТИГНУТА!")
    break

policy = new_policy

return policy, v, history

print("\n" + "-" * 40)
print("ЗАПУСК POLICY ITERATION...")
print("-" * 40)

start_time = time.time()
optimal_policy, optimal_v, history = policy_iteration_taxi(
    env, gamma=0.9, theta=1e-6, max_iter=30
)
end_time = time.time()

print("\n" + "=" * 60)
print("РЕЗУЛЬТАТЫ / RESULTS")
print("=" * 60)
print(f" Количество итераций политики / Policy iterations: {len(history['changes'])}")
print(f" Время выполнения / Time: {end_time - start_time:.2f} сек")
print(f" Среднее значение V(s) / Mean V(s): {np.mean(optimal_v):.2f}")
print(f" Максимальное значение V(s) / Max V(s): {np.max(optimal_v):.2f}")
print(f" Минимальное значение V(s) / Min V(s): {np.min(optimal_v):.2f}")

# График сходимости
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(range(1, len(history['changes']) + 1), history['changes'],
        'bo-', linewidth=2, markersize=8)
ax.set_xlabel('Policy Iteration Number / Номер итерации политики', fontsize=12)
ax.set_ylabel('Number of Policy Changes / Количество изменений', fontsize=12)
ax.set_title('Policy Iteration Convergence (Сходимость Policy Iteration)', fontsize=14)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("\n✅ Policy Iteration completed!")

```

```

# =====
# ТЕСТИРОВАНИЕ ОПТИМАЛЬНОЙ ПОЛИТИКИ
# TESTING OPTIMAL POLICY
# =====

print("=" * 60)
print("ТЕСТИРОВАНИЕ ОПТИМАЛЬНОЙ ПОЛИТИКИ")
print("TESTING OPTIMAL POLICY")
print("=" * 60)

def run_episode_taxi(env, policy, max_steps=100, render=False, verbose=True):
    """
    Запуск одного эпизода с заданной политикой
    Run one episode with given policy
    """
    state, _ = env.reset()
    total_reward = 0
    steps = 0
    success = False

    if verbose:
        print(f"\nНачальное состояние / Initial state: {state}")
        print("-" * 50)

    for step in range(max_steps):
        action = policy[state]
        action_names = ['↓(ЮГ)', '↑(СЕВЕР)', '→(ВОСТОК)', '←(ЗАПАД)', 'ВЗЯТЬ', 'ВЫСАДИТЬ']

        next_state, reward, terminated, truncated, info = env.step(action)
        total_reward += reward
        steps += 1

        if verbose:
            print(f" Шаг {step+1}: {action_names[action]} -> награда={reward}")

        if terminated:
            success = True
            if verbose:
                print(f"\n ✅ ЦЕЛЬ ДОСТИГНУТА! / GOAL REACHED!")
                print(f" Всего шагов / Total steps: {steps}")
                print(f" Общая награда / Total reward: {total_reward}")
            break

        state = next_state

    if not success and verbose:
        print(f"\n ❌ Цель не достигнута / Goal not reached (max steps)")

    return total_reward, steps, success

```

```
def evaluate_policy_taxi(env, policy, episodes=10, verbose_first=True):
    """
    Оценка политики на нескольких эпизодах
    Evaluate policy on multiple episodes
    """
    print(f"\n{'='*50}")
    print(f"ОЦЕНКА ПОЛИТИКИ / POLICY EVALUATION ({episodes} эпизодов)")
    print(f"{'='*50}")

    rewards = []
    steps_list = []
    successes = 0

    for ep in range(episodes):
        verbose = verbose_first and (ep == 0)
        if not verbose and ep == 1:
            print("\n ... (следующие эпизоды без деталей) ...")

        reward, steps, success = run_episode_taxi(env, policy, max_steps=100, verbose=verbose)
        rewards.append(reward)
        steps_list.append(steps)
        if success:
            successes += 1

    print(f"\n{'='*40}")
    print(f"РЕЗУЛЬТАТЫ / RESULTS")
    print(f"{'='*40}")
    print(f"Успешных эпизодов / Successful episodes: {successes}/{episodes} ({successes/episodes*100:.0f}%)")
    print(f"Средняя награда / Mean reward: {np.mean(rewards):.2f} ± {np.std(rewards):.2f}")
    print(f"Среднее количество шагов / Mean steps: {np.mean(steps_list):.2f}")
    print(f"Минимум шагов / Min steps: {np.min(steps_list)}")
    print(f"Максимум шагов / Max steps: {np.max(steps_list)}")

    return rewards, steps_list, successes
```

```
# Создаем новую среду для тестирования
env_test = gym.make('Taxi-v4')

# Оцениваем оптимальную политику
print("\n" + "-" * 40)
print("ОПТИМАЛЬНАЯ ПОЛИТИКА / OPTIMAL POLICY")
print("-" * 40)

rewards_opt, steps_opt, successes_opt = evaluate_policy_taxi(env_test, optimal_policy, episodes=5, verbose_first=True)

# Создаем случайную политику для сравнения
random_policy = np.random.randint(0, n_actions, n_states)

print("\n" + "-" * 40)
print("СЛУЧАЙНАЯ ПОЛИТИКА / RANDOM POLICY (для сравнения)")
print("-" * 40)

rewards_random, steps_random, successes_random = evaluate_policy_taxi(env_test, random_policy, episodes=5, verbose_first=False)

# Сравнение
print("\n" + "=" * 60)
print("СРАВНЕНИЕ / COMPARISON")
print("=" * 60)

print("\n{<30} {>15} {>15} {>15}".format(
    "Policy", "Success Rate", "Mean Reward", "Mean Steps"
))
print("-" * 75)
print("{<30} {>14.0f}% {>15.2f} {>15.2f}".format(
    "Optimal Policy", successes_opt/5*100, np.mean(rewards_opt), np.mean(steps_opt)
))
print("{<30} {>14.0f}% {>15.2f} {>15.2f}".format(
    "Random Policy", successes_random/5*100, np.mean(rewards_random), np.mean(steps_random)
))

print("\n" + "=" * 60)
print("ВЫВОДЫ / CONCLUSIONS")
print("=" * 60)
print("✓ Алгоритм Policy Iteration успешно нашел оптимальную стратегию для Taxi-v4")
print("✓ Оптимальная политика успешно доставляет пассажира в пункт назначения")
print("✓ Оптимальная политика показывает значительно лучшие результаты, чем случайная")
print("✓ Количество итераций: 16")
print("✓ Время выполнения: 1.53 секунды")
```